

PRIVACY POLICY

=====

Document status: Active proof of concept and development state.

DevBox already has a functioning local launcher, a graphical main interface, modular structure views, master-data and documentation maintenance, documentation snapshots, local tool detection, a repository page, and initial building blocks for the DevBox-specific publication process.

The deskNode area is actively integrated into DevBox. Supervisor and daemon processes can be started and stopped from the GUI, and their console output is displayed in the deskNode log. The product version can be edited through master data. UX themes can be named, created, deleted, duplicated, renamed, and saved. Saving a theme refreshes the deskNode product data.

In the current proof of concept, deskNode has a coupled worker architecture. Tapo, FRITZ!DECT, and Shelly are discovered and monitored locally through their own venmods; supported devices process switching states and power values. The daemon synchronizes venmod data into a global device inventory, applies an initial safety read cycle, and opens the main interface only after the initial global synchronization. Structure management supports multiple building roots, one complete device pool per building, and additional additive assignments to rooms or other structure members.

Symbol management is available as a functional catalog interface. It manages deskNode device categories and consumer devices through selection menus and dialogs for creation, editing, and deletion. New devices receive one PNG source, stored under their record ID as `symbol_source_<record_id>.png`. After every successful catalog change, `create_manufacturer_db.py` is executed so `mnfctr_db.r0b` contains current theme, category, and device data.

The graphics-pack build is functional as a proof of concept. It creates prepared consumer-graphics states from symbol sources and UX themes. The current deskNode runtime already uses language, theme, and graphics data; asset selection, broader symbol coverage, and additional visual states are still being developed.

DevBox and deskNode are not finished end-user or release versions. Credential storage, long-term stability with larger device inventories, vendor-specific edge cases, testing, packaging, and cross-platform portability remain open development tasks.

First publication: Responsible party / publisher: Markus Walloner Name: Markus Walloner Country: Germany (DE)

1. Subject Matter

This privacy policy applies to the software or project described in the accompanying project documentation.

Short Description: DevBox is the local development hub of the CYXnTrol Development Platform. Its PySide6 interface brings together project structure, master data,

documentation, local tools, product modules, repository preparation, and recurring development workflows.

With deskNode, an active product module is integrated: a local device and structure hub for smart plugs and connected loads. deskNode is extended through replaceable venmods for supported vendors and protocols.

DevBox is not an end-user product. It is an internal working environment in which requirements, data models, UX decisions, interfaces, builds, and publication states are prepared and reviewed in a traceable way.

2. Privacy Policy

PRIVACY POLICY – DRAFT VERSION

1. Purpose and scope

This privacy notice describes the currently intended local operation of DevBox. DevBox is a local development environment. Under its current concept, it is not designed to use telemetry, advertising, analytics services, or proprietary cloud services without explicit user action.

2. Controller and contact

The controller for the DevBox development state is Markus Walloner within CYX Labs. Questions about this project can be sent through the contact channel stated in the relevant repository or accompanying project documentation.

3. Locally processed data

DevBox processes the following data locally in particular:

- paths to the project root, project files, and local tools;
- contents of local SQLite databases, including product, manufacturer, documentation, repository, structure, UX-theme, language, category, and consumer metadata;
- local PNG sources for consumer devices, derived deskNode product databases, and prepared graphics packages;
- runtime and error logs in logfile.r0b as well as product-specific console output;
- detected paths to Inkscape and GIMP in locdata.r0b;
- version numbers, theme names, RGBA values, category names, device keys, and further interface settings entered by the user;
- deskNode device inventory, local network addresses, device states, power values, structure assignments, and time-based power logs;
- credentials for vendor paths where they are required for a user-triggered local device login;

- commit text as well as image paths and image files selected for explicitly triggered repository operations;
- technical output from local subprocesses, for example Python, Git, Inkscape, GIMP, daemon, worker, or product-specific scripts.

This data is generally processed locally in project folders, temporary workspaces, or under AppData and is used to perform the DevBox or deskNode function selected by the user.

4. deskNode, local network communication, and credentials

deskNode communicates in the current proof of concept with supported devices or gateways on the local network, for example through Tapo, FRITZ!DECT, or Shelly paths. Local IP addresses, device identifiers, status values, and power values may be processed. Credentials are not output in console logs or intended worker events. However, the current development state does not yet have a cross-platform encrypted credential vault. Locally persisted credentials must therefore not be treated as a hardened secret store; a secure vault solution is required before public or productive use.

5. Temporary workspaces

For documentation exports, graphics builds, repository preparation, deskNode runtime, and similar operations, DevBox creates unique temporary work folders. Copies of project files, intermediate data, graphics, masks, generated documents, runtime data, and prepared assets may be processed there. The application attempts to remove these workspaces after completion. Failed cleanup may result in individual temporary files remaining until a later manual or system cleanup.

6. External programs

DevBox may start local programs such as Inkscape, GIMP, Git, Git Credential Manager, or installers after explicit user action. Once such a program starts, its own data processing and terms may additionally apply. DevBox does not store Git passwords or access tokens as its own repository credentials.

7. Repository operations and data transfer

When a push-to-Git operation is explicitly triggered, Git transmits the repository content selected by the user, commit information, and technically required connection data to the relevant repository provider. The specific recipient, repository visibility, and selected branch result from the repository configuration maintained by the user. The user is responsible for not publishing personal or confidential data unintentionally.

8. Legal basis and review

This text is a technical draft and not individual legal advice. Before public or commercial distribution, it must be reviewed and adapted to actual data flows, contact details, potential legal bases, processor arrangements, international transfers, and applicable data-protection law.

3. Project Context

Purpose: DevBox is intended to turn recurring and error-prone development work into traceable local tools. This includes maintaining product and manufacturer data, structured documentation, prepared publication steps, integration of local creative tools, and controlled execution of product-specific development functions.

The application provides a common working foundation for products of the CYXnTrol Development Platform. Instead of keeping workflows only as memory, folder convention, or a collection of console commands, they are retained as data, scripts, GUI functions, and verifiable process chains.

For deskNode, DevBox additionally serves as a development and configuration interface for product versions, UX themes, device categories, consumer symbols, language resources, and prepared graphics packages. The deskNode runtime is intended to receive reproducible data and prepared assets from this process. The venmod architecture should also allow new local device paths to be added without unnecessarily rebuilding the daemon or already working vendor paths.

Configuration: Configuration is separated into several layers.

Project root: The project root is found through the .root file. It is the authoritative source for scripts, resources, form archives, installers, global fonts, and the central DevBox database.

Central DevBox master data: resources/organization/devbox_db.r0b contains manufacturer, product, documentation, structure, repository, UX-theme, and symbol-catalog information. The ux-deskNode table contains named theme records with record_id and theme_name. The desknode_consumer_device_categories and desknode_consumer_devices tables contain persistent technical categories and consumer devices. Schema extensions should retain existing records and add missing fields through migration.

deskNode product data: applications/deskNode/data/mnfctr_db.r0b contains master_data, ux_themes, consumer_device_categories, and consumer_devices. applications/deskNode/data/lan.r0b contains the language_package. fonts.r0b and graphic_items.r0b are product archives for runtime use. create_manufacturer_db.py refreshes the derived manufacturer database after successful version, theme, category, or consumer-device changes.

deskNode AppData and runtime: The local AppData path is built from master_data, currently typically %APPDATA%/CYX Labs/CYXnTrol/deskNode. settings.r0b stores the selected UI language and active UX theme as well as runtime information needed by the supervisor. devices.r0b contains the global device inventory and additive structure assignments. log_device_power.r0b logs resolved power values. Every active daemon run also has a unique temporary folder containing runtime_state.r0b and separate temporary venmod databases.

Device structure: A fresh installation receives at least one building root. Every building has an internal, fixed device pool as its complete building-wide overview. Devices can remain in the pool and also be assigned to spatial or functional members. Structure management supports multiple independent building roots.

Venmods: A venmod is found through coupler.py and must report a valid coupling contract. This describes its name, worker start, temporary database, device table, and upstream and downstream column mappings. The daemon validates these values before starting a worker. Tapo, FRITZ!DECT, and Shelly are the currently integrated local paths.

Credentials and security status: Credentials are required only by vendor paths that need login. They are not written to logs. The current proof of concept does not yet have a cross-platform encrypted credential vault; locally persisted development data must therefore not be treated as a hardened secret store. The planned vault solution is intended as a shared platform component rather than a venmod-specific special case.

Interface design: Colors are stored as eight-digit RGBA hexadecimal values without #, for example 00e4ffff. Font files are read recursively from resources/fonts and stored as project-root-relative paths. Font roles include large headlines, section headlines, body text, button text, input text, status messages, and log text. Further theme values cover font sizes, font styles, underlining, outlines, radii, and state colors.

Tool detection and repository data: Stored paths are checked at startup. Invalid Inkscape or GIMP paths are searched again. Repository URL and branch are maintained per product in product_credentials. Git credentials are not stored in DevBox. For a DevBox push, a temporary publication root is created; the current export scope is defined by the controlled push process.

Technology: DevBox currently uses the following technologies in particular:

- Python as the core language for launchers, process logic, data preparation, automation, and product-specific tools.
- PySide6 for the local graphical interfaces of DevBox and deskNode.
- SQLite for central project master data, deskNode product data, and local runtime, device, settings, language, and log data.
- python-kasa for local Tapo discovery and device communication in the Tapo venmod.
- Local HTTP, XML, and RPC communication for FRITZ!DECT/AHA and Shelly paths where implemented by the respective venmod.
- A local worker-control hub and separate worker processes for venmod integration.
- openpyxl as a transitional or import tool for spreadsheets, not as the central master-data source.
- ReportLab and svglib for local generation of designed PDF documents.
- Git and Git Credential Manager for repository operations and authentication.
- Inkscape for scaling, SVG work, and vectorization of masks.
- GIMP 3 for non-interactive image processing and preparatory mask stages.
- XML/SVG processing through Python standard libraries for cleanup of generated masks.
- C# generation, the .NET SDK, WiX, and later additional packaging tools for build and installer processes.

- temporary workspaces under the system temporary directory for controlled copies, graphics builds, exports, daemon runtimes, and publication preparation.

The technical structure aims to clearly separate GUI, function scripts, subscripts, data sources, product databases, external tools, global device storage, and venmod-specific runtime. The deskNode symbol catalog links SQLite records to PNG sources through stable record_id values, while the graphics-pack build creates the resulting visual variants.

Repository Note: The DevBox repository represents the CYXnTrol Development Platform itself, not a finished end-user product. It serves as a development state, transparent work sample, and foundation for maintaining the platform.

The published state is not copied directly from the productive project root. DevBox generates a separate cleaned root_dir state. It contains intended source code, documents, and assets, while local runtime data, temporary leftovers, installers, database WAL files, and unpublished data remain outside the publication package.

The current DevBox push treats applications as a deliberate delivery window: the area is cleaned first. Afterwards, applications/deskNode/resources/scripts including contained files are copied into the publication state. __pycache__ folders and .pyc and .pyo files remain excluded. This current scope is a DevBox-specific publication process and not yet a complete deskNode product release. Before deskNode including GUI, logic, and venmods is published as its own repository or release, its export scope must be explicitly defined and tested.

DevBox is developed through an AI-assisted, iterative workflow. Requirements, architecture, data models, UX decisions, test cases, and acceptance are directed by the project owner. Implementation is created with AI-assisted development tools and checked against defined functional requirements.

Copyright (c) 2026 Markus Walloner